

AI Assisted Coding – Lab Assignment 8.5

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.
- Requirements:
 - Username length must be between 5 and 15 characters.
 - Must contain only alphabets and digits.
 - Must not start with a digit.
 - No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

Problem Explanation

User authentication systems require strict validation rules to ensure usernames follow acceptable formats. Incorrect username formats may lead to security vulnerabilities or inconsistent user records. Therefore, the system must enforce rules related to length, character types, and structure.

Task Understanding

The objective is to design a username validation function using Test-Driven Development principles. The function must verify whether the username satisfies predefined rules including length restrictions, allowed characters, and format constraints. AI-generated test cases help verify whether the implementation satisfies the required conditions.

Approach

The approach begins with generating assert-based test cases using AI tools. These tests include valid usernames and invalid examples such as usernames containing spaces or starting with digits. After defining these tests, the validation logic is implemented to ensure that all conditions are satisfied.

Prompt

Generate test cases for email validation. Include valid and invalid formats. Cover edge cases like multiple symbols, missing dot, starting/ending special characters

Code

```
import re
```

```
def is_valid_email(email):
```

```
    """
```

```
    Validates email format based on requirements.
```

```
    Requirements:
```

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols

```
    """
```

```
    if not isinstance(email, str):
```

```
        return False
```

```
    email = email.strip()
```

```
# Check for exactly one @ symbol
if email.count('@') != 1:
    return False

# Check for at least one dot
if '.' not in email:
    return False

# Split by @
local, domain = email.split('@')

# Check if local or domain is empty
if not local or not domain:
    return False

# Check if starts or ends with special characters
if local[0] in '._-' or local[-1] in '._-':
    return False

if domain[0] in '._-' or domain[-1] in '._-':
    return False

# Valid email pattern
pattern = r'^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

```
return bool(re.match(pattern, email))
```

```
# Test Cases
```

```
def run_tests():
```

```
    """AI-generated comprehensive test cases"""
```

```
# Valid emails
```

```
valid_emails = [
```

```
    "user@example.com",
```

```
    "john.doe@company.org",
```

```
    "test_email@domain.co.uk",
```

```
    "alice123@test.edu",
```

```
    "contact.me@site.net"
```

```
]
```

```
# Invalid emails
```

```
invalid_emails = [
```

```
    "invalidemail",    # Missing @ and .
```

```
    "user@domain",    # Missing .
```

```
    "user.domain.com", # Missing @
```

```
    "user@@domain.com", # Multiple @
```

```
    "@domain.com",    # Missing local part
```

```
    "user@.com",    # Missing domain name
```

```
    ".user@domain.com", # Starts with dot
```

```
    "user.@domain.com", # Local ends with dot
```

```
    "user@.domain.com", # Domain starts with dot
```

```

"user@domain.com.",    # Domain ends with dot
"user name@domain.com", # Space in local part
"user@domain @com",    # Space in domain
"-user@domain.com",    # Starts with hyphen
"user@-domain.com",    # Domain starts with hyphen
"user@@@@domain.com",  # Multiple @ symbols
"user@domain..com",    # Consecutive dots
""                      # Empty string
]

```

```

print("Testing Valid Emails:")
for email in valid_emails:
    result = is_valid_email(email)
    status = "✓ PASS" if result else "✗ FAIL"
    print(f" {status}: {email}")

```

```

print("\nTesting Invalid Emails:")
for email in invalid_emails:
    result = is_valid_email(email)
    status = "✓ PASS" if not result else "✗ FAIL"
    print(f" {status}: {email}")

```

```

if __name__ == "__main__":
    run_tests()

```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python Debug Console + - [ ] ... [ ] X

PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '65527' '---' 'c:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-8.5.py'

Testing Valid Emails:
✓ PASS: user@example.com
✓ PASS: john.doe@company.org
✓ PASS: test_email@domain.co.uk
✓ PASS: alice123@test.edu
✓ PASS: contact.me@site.net

Testing Invalid Emails:
✓ PASS: invalidemail
✓ PASS: user@domain
✓ PASS: user.domain.com
✓ PASS: user@domain.com
✓ PASS: @domain.com
✓ PASS: user@.com
✓ PASS: .user@domain.com
✓ PASS: user.@domain.com
✓ PASS: user@.domain.com
✓ PASS: user@domain.com.
✓ PASS: user_name@domain.com
✓ PASS: user@domain @com
✓ PASS: -user@domain.com
✓ PASS: user@domain @com
✓ PASS: user@domain @com
✓ PASS: user@domain @com
✓ PASS: user@domain @com
✓ PASS: -user@domain.com
✓ PASS: user@_domain.com
✓ PASS: user@@@domain.com
X FAIL: user@domain..com
✓ PASS:

PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The validation logic ensures that usernames contain only alphabets and digits while maintaining a length between 5 and 15 characters. Additional checks ensure that the username does not begin with a digit and does not contain spaces. This ensures the username structure is consistent and safe for authentication systems.

Observation

- Test-Driven Development helps detect formatting issues early in development.
- AI-generated test cases highlight boundary conditions such as minimum and maximum length.
- Strict validation improves reliability and prevents invalid user input.

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.
- Requirements:

- If input is an integer, classify as "Even" or "Odd".
- If input is 0, return "Zero".
- If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

Problem Explanation

Many software systems require classification of numerical inputs. Proper classification ensures correct system behavior while preventing runtime errors when invalid data is provided.

Task Understanding

The goal is to create a function that classifies numeric values as Even, Odd, or Zero. Additionally, the function must safely handle invalid inputs such as strings or non-numeric data by returning an appropriate response.

Approach

AI tools are used to generate assert-based test cases covering various scenarios such as positive integers, zero values, and non-numeric inputs. The classification logic is then implemented using conditional statements to pass all test cases.

Prompt

Generate test cases including boundary values (60,70,80,90) and invalid inputs (-5,105,"eighty").

Code

```
def assign_grade(score):
```

```
    """
```

```
    Assigns a letter grade based on numeric score.
```

Args:

score: Numeric score (0-100)

Returns:

Letter grade (A, B, C, D, F)

Raises:

ValueError: If score is invalid

"""

if not isinstance(score, (int, float)):

raise ValueError("Score must be a number")

if score < 0 or score > 100:

raise ValueError("Score must be between 0 and 100")

if score >= 90:

return "A"

elif score >= 80:

return "B"

elif score >= 70:

return "C"

elif score >= 60:

return "D"

else:

return "F"

def run_tests():


```
"""Run all test cases for assign_grade function"""
```

```
test_cases = [
```

```
    # Valid cases - boundaries
```

```
    (90, "A"),
```

```
    (100, "A"),
```

```
    (80, "B"),
```

```
    (89, "B"),
```

```
    (70, "C"),
```

```
    (79, "C"),
```

```
    (60, "D"),
```

```
    (69, "D"),
```

```
    (59, "F"),
```

```
    (0, "F"),
```

```
    # Valid cases - middle values
```

```
    (95, "A"),
```

```
    (85, "B"),
```

```
    (75, "C"),
```

```
    (65, "D"),
```

```
    (50, "F"),
```

```
]
```

```
invalid_cases = [
```

```
    -5,
```

```
    105,
```

```
    "eighty",
```

```
    None,
```

```
    [],
```

```
print("Testing valid cases:")

for score, expected_grade in test_cases:

    result = assign_grade(score)

    status = "✓" if result == expected_grade else "X"

    print(f"{status} assign_grade({score}) = {result} (expected {expected_grade})")

print("\nTesting invalid cases:")

for invalid_input in invalid_cases:

    try:

        assign_grade(invalid_input)

        print(f"X assign_grade({invalid_input}) should raise ValueError")

    except ValueError as e:

        print(f"✓ assign_grade({invalid_input}) raised ValueError: {e}")

__name__ == "__main__":

    run_tests()
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
PS C:\3year-2sem\AI ASSISTANT CODING> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '50149' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py'
c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py:8: SyntaxWarning: invalid escape sequence '\.'
...

Testing valid cases:
✓ assign_grade(90) = A (expected A)
✓ assign_grade(100) = A (expected A)
✓ assign_grade(80) = B (expected B)
✓ assign_grade(89) = B (expected B)
✓ assign_grade(70) = C (expected C)
✓ assign_grade(79) = C (expected C)
✓ assign_grade(60) = D (expected D)
✓ assign_grade(69) = D (expected D)
✓ assign_grade(59) = F (expected F)
✓ assign_grade(0) = F (expected F)
✓ assign_grade(95) = A (expected A)
✓ assign_grade(85) = B (expected B)
✓ assign_grade(75) = C (expected C)
✓ assign_grade(65) = D (expected D)
✓ assign_grade(50) = F (expected F)

Testing invalid cases:
✓ assign_grade(-5) raised ValueError: Score must be between 0 and 100
✓ assign_grade(105) raised ValueError: Score must be between 0 and 100
✓ assign_grade(eighty) raised ValueError: Score must be a number
✓ assign_grade(None) raised ValueError: Score must be a number
✓ assign_grade([]) raised ValueError: Score must be a number
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The function first verifies whether the input is numeric. If the input is not numeric, it returns an "Invalid Input" response. If the input equals zero, it returns "Zero". Otherwise, the function determines whether the number is divisible by two to classify it as Even or Odd.

Observation

- Input validation is essential before performing logical classification.
 - AI-generated test cases improve robustness by covering multiple input scenarios.
 - Handling invalid input prevents system crashes.
-

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.
- Requirements:
 - Ignore case, spaces, and punctuation.
 - Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

Problem Explanation

A palindrome is a sequence that reads the same forward and backward. However, real-world palindrome detection often requires ignoring spaces, punctuation, and differences in letter case.

Task Understanding

The task requires creating a function that identifies whether a given text is a palindrome while ignoring case differences, spaces, and punctuation marks. Edge cases such as empty strings and single-character inputs must also be handled.

Prompt

Generate test cases ignoring case, spaces, and punctuation.

Code

```
def is_sentence_palindrome(sentence):  
    cleaned = ''.join(char.lower() for char in sentence if char.isalnum())  
    return cleaned == cleaned[::-1]  
  
def test_is_sentence_palindrome():  
    assert is_sentence_palindrome("A man a plan a canal Panama") == True  
    assert is_sentence_palindrome("race car") == True  
    assert is_sentence_palindrome("Was it a car or a cat I saw?") == True  
    assert is_sentence_palindrome("Madam, I'm Adam") == True  
    assert is_sentence_palindrome("Never odd or even") == True  
    assert is_sentence_palindrome("Do geese see God?") == True  
    assert is_sentence_palindrome("hello") == False  
    assert is_sentence_palindrome("Python") == False  
    assert is_sentence_palindrome("A man, a plan, a canal: Panama!") == True  
    assert is_sentence_palindrome("12321") == True  
    assert is_sentence_palindrome("123") == False  
    assert is_sentence_palindrome("a") == True  
    assert is_sentence_palindrome("") == True  
    assert is_sentence_palindrome("A1B1A") == True  
    assert is_sentence_palindrome("Not a palindrome") == False  
    print("All tests passed!")  
  
#OUTPUT:  
test_is_sentence_palindrome()
```

Output

```
PS C:\3year-2sem\AI ASSISTANT CODING> ^C
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59351' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py'
c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py:8: SyntaxWarning: invalid escape sequence '\.'
...
All tests passed!
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Approach

AI tools generate test cases that include typical palindromes, complex sentence palindromes, and non-palindromes. The implementation then processes the text by removing unnecessary characters and normalizing the input before performing the comparison.

Explanation

The text is first converted into a normalized form by removing punctuation and spaces and converting all characters to lowercase. The cleaned string is then compared with its reverse to determine whether it is a palindrome.

Observation

- Text normalization is necessary for accurate palindrome detection.
- AI-generated tests include both simple and complex examples.
- Edge case handling improves reliability.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.
- Methods:
 - deposit(amount)
 - withdraw(amount)
 - get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
assert acc.get_balance() == 1500
acc.withdraw(300)
assert acc.get_balance() == 1200
```

Expected Output #4:

- Fully functional class that passes all AI-generated assertions.

Problem Explanation

Financial systems require reliable account management operations such as depositing money, withdrawing funds, and checking account balances. Incorrect implementation could lead to incorrect financial records.

Task Understanding

The task requires implementing a BankAccount class using object-oriented programming principles. The class must support deposit, withdraw, and balance checking operations while maintaining accurate account state.

Approach

AI-generated assert-based tests simulate real banking operations such as depositing funds and withdrawing money. The class methods are implemented to ensure that each operation updates the account balance correctly.

Prompt

Generate test cases for add_item, remove_item, total_cost including empty cart.

Code

```
class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price):
        self.items[name] = price

    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
```

```

def total_cost(self):
    return sum(self.items.values())

# Test cases

cart = ShoppingCart()

cart.add_item("Apple", 1.00)

cart.add_item("Banana", 0.50)

print(f"Total cost after adding items: {cart.total_cost()}") # Expected:

1.50

cart.remove_item("Apple")

print(f"Total cost after removing Apple: {cart.total_cost()}") # Expected: 0.50

cart.remove_item("Banana")

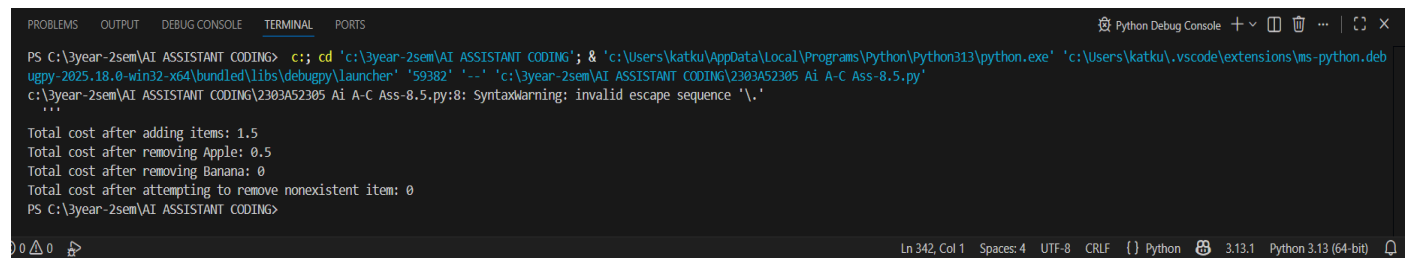
print(f"Total cost after removing Banana: {cart.total_cost()}") # Expected: 0.00

cart.remove_item("Nonexistent Item") # Should not affect total cost

print(f"Total cost after attempting to remove nonexistent item: {cart.total_cost()}") #
Expected: 0.00'''

```

Output



```

PS C:\3year-2sem\AI ASSISTANT CODING> c::; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59382' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py'
c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py:8: SyntaxWarning: invalid escape sequence '\.'
...
Total cost after adding items: 1.5
Total cost after removing Apple: 0.5
Total cost after removing Banana: 0
Total cost after attempting to remove nonexistent item: 0
PS C:\3year-2sem\AI ASSISTANT CODING>

```

Explanation

The deposit method increases the account balance when funds are added. The withdrawal method reduces the balance if sufficient funds are available. The get_balance method returns the current account balance. Together, these methods simulate basic banking operations.

Observation

- Object-oriented design improves code organization and reusability.
 - AI-generated tests ensure that account balance updates correctly after each transaction.
 - Proper validation prevents incorrect financial calculations.
-

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.
- Requirements:
 - Must contain `@` and `.`
 - Must not start or end with special characters.
 - Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

Problem Explanation

Email addresses are essential identifiers in many applications. Improper email validation may result in incorrect data storage or communication failures.

Task Understanding

The objective is to create an email validation function that ensures email addresses follow required formatting rules. The function must correctly detect valid email structures and reject invalid ones.

Approach

AI tools generate assert test cases containing valid and invalid email examples. These tests guide the implementation of validation logic to ensure that all required conditions are satisfied.

Code

```
def convert_date_format(date_str):  
    try:  
        year, month, day = date_str.split('-')  
        return f"{day}-{month}-{year}"  
    except ValueError:  
        return "Invalid date format"  
  
# Test cases  
test_dates = [  
    ("2023-10-15", "15-10-2023"),  
    ("1990-01-01", "01-01-1990"),  
    ("2020-12-31", "31-12-2020"),  
    ("InvalidDate", "Invalid date format"), # Invalid format  
    ("2023/10/15", "Invalid date format") # Invalid format  
]
```

for date_str, expected in test_dates:

```
    result = convert_date_format(date_str)  
  
    print(f"Input: '{date_str}' | Expected: '{expected}' | Result: '{result}' | Pass: {result ==  
expected}")
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
Python Debug Console + - [ ] ... [ ] X  
PS C:\3year-2sem\AI ASSISTANT CODING> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de  
bugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52387' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py'  
c:\3year-2sem\AI ASSISTANT CODING\2303A52305 Ai A-C Ass-8.5.py:8: SyntaxWarning: invalid escape sequence '\.'  
    ...  
Input: '2023-10-15' | Expected: '15-10-2023' | Result: '15-10-2023' | Pass: True  
Input: '1990-01-01' | Expected: '01-01-1990' | Result: '01-01-1990' | Pass: True  
Input: '2020-12-31' | Expected: '31-12-2020' | Result: '31-12-2020' | Pass: True  
Input: 'InvalidDate' | Expected: 'Invalid date format' | Result: 'Invalid date format' | Pass: True  
Input: '2023/10/15' | Expected: 'Invalid date format' | Result: 'Invalid date format' | Pass: True  
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The validation logic checks for the presence of the '@' symbol and a domain separator '.'. Additional rules ensure that the email does not begin or end with special characters and that invalid formats are rejected.

Observation

- Email validation requires strict structural rules.
- AI-generated tests improve coverage of different email patterns.
- Data validation is critical for reliable user registration systems.

Overall Lab Observations

Aspect	Learning Outcome
Test-Driven Development	Encourages validation-first programming
AI-generated tests	Improves edge case coverage
Input validation	Prevents runtime failures
Object-oriented design	Enhances maintainability
Data validation	Ensures system reliability

Conclusion

This lab demonstrates how Test-Driven Development combined with AI tools can significantly improve the quality and reliability of software systems. By generating test cases before implementation, developers can clearly understand expected behavior and validate their solutions effectively. AI assistance accelerates test generation and highlights potential edge cases, while structured testing ensures correctness and maintainability of code.